

OMI Doctrine: The Rewrite Geometry of Computation

First Principle

OMI does not begin with data.

OMI begins with interpretation.

Traditional computing treats memory as data stored at an address:

```
RAM[address] = value
```

OMI treats memory as a rewrite relationship:

```
OMI[orbit,address] = rewrite
```

The fundamental operation is not storage.

The fundamental operation is transformation.

The Omicron

The symbol O (Omicron) is not merely a character.

It is a notation selector.

Like the sign bit in floating-point arithmetic, it determines how a structure is interpreted.

In IEEE-754:

```
sign  
exponent  
significand
```

In OMI:

Omicron
address
value

The Omicron is not positive or negative.

The Omicron selects a mode of interpretation.

This is why:

omi-

and

-imo

are dual forms.

One describes the addressing perspective.

One describes the interpretation perspective.

The data remains unchanged.

Only the interpretation changes.

Control Codes Are Not Characters

The ASCII control range:

0x00 . . 0x1F

is usually described as thirty-two non-printing characters.

OMI treats them differently.

They are rewrite operators.

Examples include:

NUL
SOH
STX
ETX
EOT
ENQ
ACK
BEL
BS
HT
LF
CR
SO
SI

Historically these were protocol operations.

They were never intended to be symbols.

They described transitions.

OMI restores this interpretation.

Control codes become transformation operators rather than textual characters.

Printable Characters Are Projections

The printable ASCII range:

0x20 . . 0x7F

is not viewed as isolated symbols.

Instead it is the visible projection of rewrite operations acting on a finite address space.

The visible character is only the displayed form.

The rewrite relation is the underlying structure.

The 4-Bit Wheel

The smallest complete address wheel is:

0x0 . . 0xF

or:

16 states

OMI interprets this as:

15 active states
1 closure state

The closure state is:

0000

which functions as a projective center.

Thus:

$16 \rightarrow 15 + 1$

This produces the canonical:

15-of-16

construction.

The 8-Bit Wheel

Likewise:

$$2^3 = 8$$

becomes:

7 active states
1 closure state

producing:

7-of-8

This is why seven-state rotational structures repeatedly appear throughout OMI.

The Fano plane emerges naturally from this construction.

Rotation Instead of Shifting

Traditional systems frequently use shifts.

Shifts discard information.

OMI uses rotations.

A rotation preserves information.

Nothing falls off the edge.

The state simply moves around the wheel.

The canonical transition law is:

$$\Delta C(x) = \text{rotl}(x,1) \oplus \text{rotl}(x,3) \oplus \text{rotr}(x,2) \oplus C$$

where:

C

is the closure variable carried forward through the orbit.

The orbit therefore remembers its previous state.

The Closure Variable

The variable:

C

is not merely a constant.

It is the carried closure of the current orbit.

Every orbit leaves behind a witness.

Every rewrite contributes to future rewrites.

This is why OMI does not view memory as passive storage.

Memory is accumulated closure.

Unicode As Rewrite Space

Unicode is usually described as a character set.

OMI interprets Unicode as a rewrite manifold.

The Unicode planes are not merely storage regions.

They are rewrite regions.

Particularly significant are the private-use areas:

U+E000–U+F8FF
U+F0000–U+FFFFD
U+100000–U+10FFFD

which provide bounded regions suitable for protocol-level interpretation without conflicting with public character assignments.

The Incidence View

An ASCII table is usually taught as a lookup table.

OMI treats it as an incidence system.

Rows are blocks.

Code points are points.

Relationships are incidences.

Thus the character table becomes a finite geometry.

The table is no longer merely a lookup structure.

It becomes a rewrite structure.

The Projective Point

Traditional geometry introduces points at infinity.

OMI introduces closure points.

The closure point is:

0x0 . . 0xF

viewed as a projective center.

When rotational address spaces collapse into closure, they meet at a common center.

This produces a projective interpretation of finite computation.

The closure point is not outside the system.

It is the self-reference of the system.

The Empty List

Lisp reveals an important computational truth.

The empty list:

```
()
```

is simultaneously:

```
an atom  
and  
a list
```

OMI treats closure similarly.

The closure state is simultaneously:

```
a point  
and  
a space
```

It is both origin and container.

This dual role is fundamental to recursive computation.

The Rewrite Doctrine

OMI does not describe computation as:

```
data changing
```

OMI describes computation as:

```
interpretation changing
```

The bits remain.

The orbit changes.

The address remains.

The rewrite changes.

The symbol remains.

The meaning changes.

The system is therefore best understood not as a memory architecture but as a rewrite geometry.

Everything else follows from this principle.